Sumário

1. Sumário Executivo 2

2. Verificação Atual da Aplicação (v2.6.0) 2

3. O Fator Crítico: Síntese de Voz por Camada 3

4. Android — Análise de Rotas Nativas 4

4.1 TWA (Trusted Web Activity) via Bubblewrap — rota recomendada 4

4.2 Capacitor com plugin nativo de TTS — alternativa autossuficiente 5

4.3 e 4.4 Reescrita nativa (Kotlin/Compose e Flutter) — não recomendadas 5

5. Windows 10/11 — Análise de Rotas Nativas 6

5.1 Electron — rota recomendada 6

5.2 Tauri 2 com WebView2 — otimização futura 6

5.3 PWA publicada na Microsoft Store — complemento de baixo custo 6

5.4 Reescrita nativa em WinUI 3 — não recomendada 6

6. Conformidade Infantil e Requisitos das Lojas 7

7. Custos, Contas e Pré-requisitos 8

8. Recomendação e Veredito 9

9. Roadmap de Implementação em Fases 9

10. Riscos e Mitigações 10

11. Conclusão 11

1. Sumário Executivo

Este estudo avalia a viabilidade técnica e operacional de produzir duas versões nativas do jogo Conta Comigo: uma para **Android** (pacote APK/AAB, apto ao Google Play e à distribuição direta) e uma para **Windows 10/11** (executável e/ou pacote MSIX, apto à Microsoft Store e à distribuição direta). A conclusão central é positiva: **ambas as versões são viáveis sem reescrever nenhuma linha do jogo**. Como todo o gameplay já roda em JavaScript puro, sem dependências externas e com persistência local, as rotas recomendadas consistem em embrulhar a aplicação existente em camadas nativas maduras, mantendo o motor do jogo intacto como base única de código.

Para Android, a rota recomendada é a **TWA (Trusted Web Activity)** gerada pelo Bubblewrap, ferramenta oficial do Google, sobre a PWA já existente hospedada publicamente. Essa rota preserva integralmente a síntese de voz — descoberta crítica desta pesquisa: a API de voz **não funciona no WebView Android** (defeito confirmado no rastreador do Chromium), o que tornaria a rota Capacitor dependente de um plugin de TTS nativo. Para Windows, a rota recomendada é o **Electron** empacotado automaticamente pelo GitHub Actions, que produz um executável autossuficiente com o Chromium embutido e voz funcionando pelas vozes do sistema.

A verificação da aplicação nesta rodada confirmou a integridade da v2.6.0: jogo funcional nas extensões e na PWA, selo de versão visível, portão dos responsáveis operante, feedback falado com o nome do perfil e nenhum erro de JavaScript. O custo inicial do plano é baixo — hospedagem gratuita no GitHub Pages, compilação gratuita no GitHub Actions e, quando chegar a hora das lojas, uma única taxa de US$ 25 do Google Play (a conta individual da Microsoft Store deixou de cobrar taxa no fluxo atual). As próximas ações concretas estão detalhadas no roadmap de quatro fases do capítulo 9.

100% do jogo aproveitado — nenhuma reescrita do gameplay é necessária

Android: VIÁVEL pela rota TWA · Windows 10/11: VIÁVEL pela rota Electron

2. Verificação Atual da Aplicação (v2.6.0)

Antes de planejar as versões nativas, a aplicação foi verificada de ponta a ponta nesta rodada, no estado atual do repositório (v2.6.0). O teste foi executado em navegador real sobre os arquivos da extensão Chrome e da PWA Android, cobrindo tanto a abertura e o fluxo de primeira execução quanto uma rodada completa de jogo. Os resultados confirmam que a base sobre a qual as camadas nativas serão construídas está sólida, estável e sem regressões.

O comportamento observado foi o esperado em todos os pontos testados. O jogo abre corretamente até por caminho local de arquivos (o mecanismo de fallback para localStorage funciona), o selo de versão v2.6.0 aparece no topo em todas as telas, o fluxo de onboarding exibe o portão dos responsáveis com criação de senha adiável, e uma rodada do modo Contar respondeu com correção gentil falada, chamando a criança pelo nome do perfil — exatamente o comportamento definido nas versões 2.4.0 e 2.4.1. Em viewport móvel de 360 pixels não houve transbordo da barra superior, confirmando o trabalho de responsividade da v2.6.0. A API de síntese de voz está presente no ambiente de teste (sem vozes instaladas, o que é normal em navegador sem interface gráfica), e

nenhum erro de JavaScript foi registrado durante toda a sessão.

Verificação executada	Resultado
Abertura do jogo (extensão, caminho file:// e PWA)	OK — carrega e persiste estado
Selo de versão no topo (v2.6.0)	OK — visível em todas as telas
Onboarding: portão dos responsáveis	OK — criação/adiamento de senha
Rodada completa de Contar com feedback falado	OK — elogio com nome do perfil
Viewport 360 px (celular pequeno)	OK — sem transbordo
API speechSynthesis presente	OK — 0 vozes no ambiente de teste (esperado)
Erros de JavaScript na sessão	Nenhum

Tabela 1 — Verificação da aplicação realizada nesta rodada (v2.6.0).

Do ponto de vista de portabilidade, a arquitetura atual é uma aliada rara: JavaScript vanilla sem etapa de build, zero dependências de bibliotecas, caminhos relativos entre arquivos, e uma camada de persistência que já abstrai a diferença entre chrome.storage e localStorage. Em outras palavras, o jogo não sabe — e não precisa saber — se roda dentro de uma extensão, de um navegador, de um WebView ou de um executável. A única área que exige atenção especial nas camadas nativas é a síntese de voz, tratada em profundidade no próximo capítulo.

3. O Fator Crítico: Síntese de Voz por Camada

O Conta Comigo é dirigido a crianças a partir de 4 anos — majoritariamente pré-leitoras. Para esse público, a narração falada não é um enfeite: é a interface principal, responsável por instruções, correções e celebrações. Qualquer rota nativa que quebre a síntese de voz quebra o jogo. Por isso, o primeiro critério de seleção de tecnologia não é tamanho de pacote nem modismo técnico, e sim: **onde a voz funciona?**

A pesquisa desta rodada confirmou um fato técnico decisivo e ainda pouco conhecido: a API Web Speech (speechSynthesis) **não está disponível no WebView do Android**. O defeito está registrado no rastreado oficial do Chromium (issue 40417848) e afeta diretamente qualquer embrulho baseado em WebView — caso do Capacitor, do Cordova e de genéricos de conversão de site em app. Felizmente, as rotas que preservam a voz existem e são justamente as mais leves: a TWA executa a PWA dentro do Chrome completo (o mesmo motor do navegador, com as vozes do Google TTS), e o Electron embute um Chromium completo no Windows, expondo as vozes de síntese do sistema (família Microsoft/SAPI, com vozes em português como a Maria). A Tauri 2, que usa o WebView2 do Windows, opera em terreno intermediário e precisa de validação prática em prova de conceito.

Camada de execução	Motor	speechSynthesis	Vozes PT-BR típicas	Impacto no jogo
Chrome/Edge desktop (extensão e PWA)	Chromium	Sim	Edge Natural (rede), Google, Microsoft	Voz plena
Firefox (extensão)	Gecko	Sim (menos vozes)	Vozes do sistema	Voz reduzida
PWA no Chrome Android	Chromium	Sim	Google TTS PT-BR	Voz plena
TWA (Bubblewrap)	Chrome completo	Sim	Google TTS PT-BR	Voz plena
Capacitor (WebView do sistema)	WebView Chromium	Não (issue 40417848)	não se aplica	Quebra sem plugin de TTS
Electron	Chromium embutido	Sim	Microsoft/SAPI (ex.: Maria)	Voz plena
Tauri 2 (WebView2)	Edge WebView2	A validar	não se aplica	Risco a validar em PoC

Tabela 2 — Disponibilidade da síntese de voz por camada de execução (pesquisa desta rodada).

A implicação prática é direta: se um dia optarmos por Capacitor (por exemplo, para integrar recursos nativos profundos), o módulo de voz do engine.js precisará de um segundo back-end via plugin nativo de TTS — um trabalho de adaptação de dias, não de semanas, porque a v2.4.0 já concentrou a fala em um único ponto de decisão com preferências por perfil. Mas, com as rotas TWA e Electron, essa adaptação se torna desnecessária na primeira fase, e a criança ouve exatamente a mesma voz que já conhece.

4. Android — Análise de Rotas Nativas

Foram avaliadas quatro rotas para produzir uma aplicação Android nativa ou nativamente distribuída: TWA via Bubblewrap, Capacitor com plugin de TTS, reescrita nativa em Kotlin e reescrita em Flutter/React Native. O critério de desempate, como explicado, é a preservação da voz; seguem-se esforço, tamanho de pacote, pré-requisitos e adequação ao programa infantil do Google Play.

4.1 TWA (Trusted Web Activity) via Bubblewrap — rota recomendada

A TWA é o mecanismo oficial do Google para publicar uma PWA na Play Store como aplicação nativa. O Bubblewrap gera um pequeno projeto Android que abre o jogo em tela cheia dentro do próprio Chrome do aparelho — sem barra de navegador e sem WebView. Como o motor é o Chrome completo, **toda a camada de voz funciona sem alterações**, inclusive o detector de vozes naturais da v2.6.0. O pacote resultante é minúsculo (2 a 4 MB, pois não carrega motor algum) e a experiência é indistinguível de um app nativo para a criança e a família.

Os pré-requisitos são documentais, não técnicos: a PWA precisa estar publicada em HTTPS (o **GitHub Pages** resolve de graça, no endereço klivfy.github.io/Conta-Comigo — o repositório já existe), com manifesto, service

worker e ícones válidos — todos já prontos na v2.6.0 — e um arquivo de verificação assetlinks.json publicado no domínio para eliminar a barra de URL. A compilação do APK/AAB pode rodar inteiramente no **GitHub Actions**, que já oferece o JDK necessário nos executores Ubuntu, dispensando qualquer instalação local. A limitação honesta da rota: como o app depende do Chrome do aparelho, a primeira abertura requer conexão para o Chrome validar a associação; depois, o service worker garante o jogo offline já existente.

4.2 Capacitor com plugin nativo de TTS — alternativa autossuficiente

O Capacitor gera um projeto Android real com o jogo embutido e um WebView do sistema como tela. Sua vantagem é ser autossuficiente: um único APK carrega tudo, sem depender de hospedagem nem do Chrome do aparelho, e abre caminho para recursos nativos profundos (notificações locais dos baús de ofensiva, por exemplo). Sua desvantagem é exatamente o fator crítico do capítulo 3: o WebView não expõe speechSynthesis, exigindo a integração do plugin comunitário de Text-to-Speech e uma pequena adaptação do módulo de voz do engine.js para escolher o back-end conforme o ambiente. O esforço estimado sobe para 8 a 12 dias e o pacote, para 8 a 15 MB. É uma rota perfeitamente viável — apenas mais cara em trabalho, e por isso posicionada como evolução, não como primeiro passo.

4.3 e 4.4 Reescrita nativa (Kotlin/Compose e Flutter) — não recomendadas

Reescrever o jogo em Kotlin/Compose ou Flutter significaria reimplementar, em outra linguagem, todo o motor pedagógico construído e validado em nove versões: geração de questões por nível, modos de jogo, duelos, figurinhas, conquistas, loja, temas, mascotes, portão dos responsáveis e a camada de voz. O esforço estimado é de 40 a 80 dias, com risco alto de regressões e de divergência entre versões. Nenhuma funcionalidade exigida pelo pedido — estar na Play Store e rodar como app Android — depende dessa reescrita. As rotas de embrulho entregam o mesmo resultado com o jogo intacto.

Critério	TWA (Bubblewrap)	Capacitor + TTS	Nativo Kotlin
Esforço estimado	3-5 dias	8-12 dias	40-80 dias
Tamanho do pacote	2-4 MB	8-15 MB	15-40 MB
Voz (speechSynthesis)	Preservada (Chrome)	Requer plugin nativo	Nativa (TextToSpeech)
Pré-requisitos	PWA hospedada + assetlinks	Adaptar camada de voz	Reescrita total
Play Store	Pronto (Play App Signing)	Pronto	Pronto
Reescrita do jogo	Nenhuma	Só a camada de voz	Total
Risco	Baixo	Médio	Alto

Tabela 3 — Matriz comparativa das rotas Android.

5. Windows 10/11 — Análise de Rotas Nativas

No Windows, o leque é diferente: não há dependência de loja de aplicativos para distribuir um executável, mas há exigências próprias — compatibilidade com Windows 10 e 11, funcionamento offline, e a barreira prática do aviso do SmartScreen para binários não assinados. Foram avaliadas quatro rotas: Electron, Tauri 2 com WebView2, PWA publicada na Microsoft Store e reescrita nativa em WinUI.

5.1 Electron — rota recomendada

O Electron embute um Chromium completo num executável Windows autossuficiente: a janela do jogo abre exatamente como no navegador, com speechSynthesis funcionando pelas vozes de síntese do Windows (família SAPI, com vozes em português instaladas por padrão no Windows 10/11). O empacotamento é o grande trunfo operacional: ferramentas consolidadas (electron-builder, electron-packager) **não exigem compilação nativa** para produzir o pacote win32, o que permite compilar no GitHub Actions a partir de um executor Linux e baixar o ZIP pronto — o mesmo padrão de CI já recomendado para o Android. O contraponto é o tamanho: 90 a 150 MB, pois cada aplicação carrega o motor. Para um jogo educativo instalado uma única vez em computadores de família, esse custo é amplamente aceitável; a otimização de tamanho pode vir depois com a Tauri.

5.2 Tauri 2 com WebView2 — otimização futura

A Tauri 2 produz binários minúsculos (5 a 10 MB) reaproveitando o WebView2, runtime do Edge já presente no Windows 10/11 atualizado. O jogo em si não muda nada. Porém, a rota exige toolchain Rust no ambiente de compilação e, sobretudo, exige validar na prática a lista de vozes exposta pelo WebView2 ao speechSynthesis — relatos da comunidade variam entre versões do runtime, e a voz é inegociável neste projeto. Por isso, a recomendação é tratá-la como otimização de segunda geração: primeiro entregar com Electron (risco baixo, voz garantida), depois, validada a Tauri em prova de conceito, migrar o embrulho se o tamanho importar.

5.3 PWA publicada na Microsoft Store — complemento de baixo custo

A Microsoft aceita publicar PWAs diretamente na loja, por URL ou convertidas em MSIX pelo PWABuilder, sem mudança de código — e o fluxo atual de 2026 **dispensou a taxa de registro para desenvolvedores individuais** (contas de organização pagam US$ 19 uma única vez). Como essa rota depende da mesma hospedagem HTTPS exigida pela TWA Android, ela sai praticamente de graça depois da Fase 1 do roadmap. É um complemento de alcance, não a rota principal: cobre quem já usa a Store, mas não substitui o executável direto para famílias que baixam fora de loja.

5.4 Reescrita nativa em WinUI 3 — não recomendada

Pelos mesmos argumentos do capítulo 4.3: meses de trabalho para replicar o que já existe, sem ganho funcional para o usuário final. Descartada.

Critério	Electron	Tauri 2 (WebView2)	PWA na Store	Nativo WinUI
Esforço	4-6 dias	6-8 dias	2-3 dias (após hospedagem)	40-80 dias
Tamanho	90-150 MB	5-10 MB	Instalação via Store	20-60 MB
Voz	Sim (Chromium + SAPI)	A validar em PoC	Sim (WebView2/Edge)	Nativa
Pré-requisitos	CI com Node	Toolchain Rust	Hospedagem + conta Store	Reescrita total
Windows 10 e 11	Sim	Sim	Sim	Sim
Reescrita do jogo	Nenhuma	Nenhuma	Nenhuma	Total
Risco	Baixo	Médio	Baixo-médio	Alto

Tabela 4 — Matriz comparativa das rotas Windows 10/11.

6. Conformidade Infantil e Requisitos das Lojas

O Conta Comigo foi desenhado desde a origem para o público infantil sob as referências COPPA, LGPD (com atenção especial a crianças) e GDPR-K: zero coleta de dados, zero telemetria, zero anúncios, zero SDKs de terceiros, com todo o progresso guardado localmente e o painel dos responsáveis protegido por verificação adulta. Essa disciplina arquitetural converte-se agora em vantagem regulatória direta: os dois programas de loja exigem exatamente o que o jogo já pratica. No Google Play, o programa Families e o formulário Data Safety pedem divulgação completa da coleta de dados — que, no nosso caso, é a afirmação mais simples possível: nenhuma. A política de privacidade exigida como URL já está escrita e publicada no repositório desde a v2.5.0.

Na Microsoft Store, a certificação pede a mesma triade: declaração de privacidade, descrição honesta do público e ausência de práticas de coleta invisíveis. Em ambos os casos, o classificatório etário (IARC no Play, questionnaire próprio na Microsoft) deve refletir o público de 4 anos, o que aciona deveres de proteção reforçada — todos já cumpridos por construção. O capítulo de riscos detalha o único cuidado real: preencher esses formulários com atenção, pois reprovações nesse estágio costumam vir de preenchimento impreciso, e não do produto.

Requisito	Google Play (Android)	Microsoft Store (Windows)
Conta de desenvolvedor	US$ 25 (pagamento único)	Sem taxa para devs individuais (fluxo 2026); organização: US$ 19
Formulário de dados	Data Safety: nenhum dado coletado	Declaração de privacidade na certificação
Programa infantil	Families Policy + questionário IARC (público de 4 anos)	Certificação padrão; restrições de anúncios
Empacotamento	AAB/APK assinado (Play App Signing)	MSIX via PWABuilder ou PWA por URL
Política de privacidade	URL obrigatória (já publicada no repositório)	URL obrigatória (a mesma)
Revisão	Testes internos imediatos; revisão em dias	Certificação automatizada em horas/dias

Tabela 5 — Requisitos de publicação nas duas lojas.

7. Custos, Contas e Pré-requisitos

O custo inicial do plano é propositalmente próximo de zero. A hospedagem da PWA usa o GitHub Pages, já incluído no repositório público que a Klivfy acabou de criar; a compilação dos pacotes roda no GitHub Actions, cuja cota gratuita cobre com folga o volume deste projeto; e a distribuição direta (APK para famílias, ZIP para Windows) não depende de nenhuma conta paga. O único gasto inevitável para estar **na** Play Store é a taxa única de US$ 25 da conta de desenvolvedor Google; a Microsoft Store, no fluxo atual para desenvolvedores individuais, não cobra registro — e, mesmo na modalidade de organização, o valor é único e simbólico. Itos opcionais ficam claramente separados: domínio próprio e certificado de assinatura de código do Windows são melhorias de credibilidade que podem chegar na Fase 3, sem bloquear nada.

Item	Custo	Papel no plano
Hospedagem da PWA (GitHub Pages)	R$ 0	Essencial para TWA e Microsoft Store
Domínio próprio (opcional)	cerca de R$ 40-60/ano	Marca própria; klivfy.github.io já funciona
Conta Google Play	US$ 25 (única vez)	Publicar na Play Store
Conta Microsoft Store (individual)	R$ 0 no fluxo 2026	Publicar na Microsoft Store
Assinatura de release Android	R$ 0 (keystore local + Play App Signing)	Obrigatória para versão de produção
Certificado de código Windows	US$ 100-400/ano (opcional)	Remove aviso do SmartScreen; Fase 3+
GitHub Actions (repositório público)	R$ 0	Compila APK e EXE automaticamente

Tabela 6 — Custos e pré-requisitos do plano de versões nativas.

8. Recomendação e Veredito

Somando os capítulos anteriores, o veredito é claro e fundamentado em dois fatos verificáveis: a voz funciona de graça nas rotas TWA e Electron, e o empacotamento de ambas pode ser automatizado no GitHub Actions sem instalações locais. A Figura 1 resume o esforço estimado de implementação por rota — as barras escuras marcam as duas rotas recomendadas, que também são as mais baratas de suas plataformas depois das reescritas descartadas.

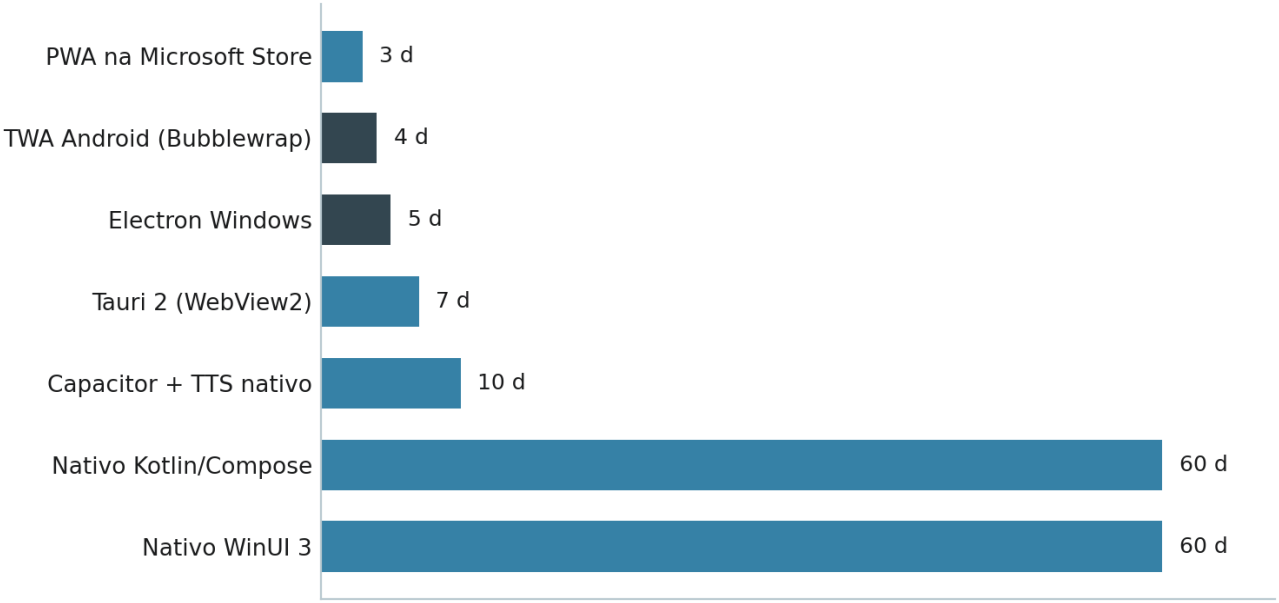


Figura 1 — Esforço estimado de implementação por rota, em dias de desenvolvimento (estimativas deste estudo).

A recomendação operacional, portanto, é uma sequência, não uma escolha única. **Primeiro**, publicar a PWA no GitHub Pages — é o pré-requisito comum da TWA e da Microsoft Store e custa uma configuração. **Segundo**, criar o embrulho TWA Android e o embrulho Electron Windows com compilação automática, entregando APK e executável testáveis ainda nesta fase. **Terceiro**, com os pacotes validados em aparelhos e computadores reais, abrir as contas das lojas e publicar. **Quarto**, otimizar: domínio próprio, Tauri para reduzir o peso do Windows e, se algum dia fizer sentido, Capacitor para recursos nativos profundos no Android — agora com a camada de voz preparada para um segundo back-end.

Plano em uma frase: PWA hospedada + TWA Android + Electron Windows, tudo compilado no GitHub Actions

9. Roadmap de Implementação em Fases

O plano está dividido em quatro fases com entregáveis verificáveis, permitindo pausas naturais entre elas. A Fase 1 é executável imediatamente neste ambiente e no repositório GitHub da Klivfy, sem nenhuma conta nova e sem custo. As fases 2 e 3 dependem, respectivamente, de aparelhos reais para validação e das contas das lojas — decisões da Klivfy Tecnologia, mas com todo o trabalho técnico precedente já pronto.

Fase	Escopo	Entregáveis verificáveis
Fase 1 — Fundação	Publicar a PWA no GitHub Pages; criar scaffolds nativos (Electron e Capacitor); configurar workflows de compilação	URL pública do jogo; APK de teste; ZIP Windows; artefatos no Actions
Fase 2 — Validação	Testar voz e jogo em Android e Windows reais; ajustar camada de voz (fallback e mapeamento de vozes PT-BR por plataforma); tela cheia e teclado	Relatório de dispositivos; engine v2.7.0 com camada de voz adaptável
Fase 3 — Lojas	Abrir contas Play e Microsoft; preencher Families, Data Safety e certificação; publicar	Apps publicados nas duas lojas; página de marca
Fase 4 — Evolução	Domínio próprio; Tauri para reduzir tamanho; integrações nativas (notificações de baú, atalhos)	Versões otimizadas e recursos nativos incrementais

Tabela 7 — Roadmap em fases com entregáveis.

10. Riscos e Mitigações

Nenhum risco identificado compromete a viabilidade; todos têm mitigação conhecida e, em geral, barata. O risco mais importante — indisponibilidade da voz no WebView Android — já está tratado estruturalmente pela escolha da rota TWA, e a mitigação da rota alternativa (plugin de TTS nativo) está mapeada com esforço estimado. Os demais riscos são operacionais e típicos de publicação: formulários das lojas, aviso do SmartScreen e verificação de domínio.

Risco	Probabilidade	Impacto	Mitigação
speechSynthesis indisponível no WebView (Capacitor)	Alta (confirmada)	Alto	Preferir TWA; com Capacitor, integrar plugin nativo de TTS na camada de voz
Lista de vozes vazia na primeira chamada (Electron)	Média	Médio	Ouvir o evento voiceschanged e re-enumerar (padrão já usado pelo detector de vozes)
Aviso do SmartScreen sem certificado de código	Alta	Baixo	Publicar na Store; orientar a instalação; certificado opcional na Fase 3+
Impedimento no Families Policy por formulário	Baixa	Alto	Zero coleta/SDKs; política de privacidade e respostas do Data Safety já preparadas
assetlinks.json incorreto (TWA sem tela cheia)	Média	Médio	Verificação automática no CI e teste em aparelho antes do release
Peso do pacote Electron (90-150 MB)	Certa	Baixo	Tauri 2 como otimização futura (5-10 MB), após validar vozes no WebView2

Tabela 8 — Riscos identificados e mitigações.

11. Conclusão

O pedido de versões nativas para Android e Windows 10/11 é **viável, barato e de baixo risco** — desde que se escolham as rotas certas, e a pesquisa desta rodada mostrou que as rotas certas são também as mais simples. A decisão técnica mais importante foi revelada pelo fator voz: a TWA Android preserva a narração porque roda no Chrome completo, enquanto caminhos baseados em WebView a quebrariam; no Windows, o Electron embute Chromium com as vozes do sistema e pode ser empacotado diretamente no GitHub Actions. Em nenhum cenário recomendado o jogo é reescrito: o motor validado até a v2.6.0 permanece como base única, e as camadas nativas são embrulhos descartáveis e substituíveis.

Os próximos passos imediatos estão no roadmap do capítulo 9 e não exigem decisão financeira: habilitar o GitHub Pages, criar os scaffolds nativos e configurar as compilações automáticas. Ao término da Fase 1, a Klivfy terá um link público do jogo, um APK instalável e um executável Windows — artefatos concretos para testar com famílias reais antes de qualquer investimento nas lojas. Recomenda-se aprovar a execução da Fase 1 e, em paralelo, agendar a validação em dispositivos reais da Fase 2, que é o único teste que nenhum ambiente simulado substitui.